

RESEARCH-ARTICLE 01 June 2026

From Technical Debt to Cognitive and Intent Debt

Rethinking software health in the age of AI

Margaret-Anne Storey

Abstract

Generative AI is dramatically accelerating software development, allowing teams to generate and modify code faster than ever before. For decades, software engineering has focused on managing technical debt—how code structure and implementation make systems harder to change. But in the age of AI, technical debt might no longer be the most important constraint. This article argues that the real risks are shifting toward two less visible forms of debt: cognitive debt and intent debt. Cognitive debt is the erosion of shared understanding across a team where no one can confidently explain how a system works or predict the impact of a change. Intent debt is the absence of clear goals, constraints, and rationale that explain what the system is for and guide how it should evolve, for both humans and AI agents. These debts have always existed, but GenAI accelerates their accumulation while hiding their effects. I propose how these forms of debt can be recognized in practice and suggest strategies teams can use to mitigate them.

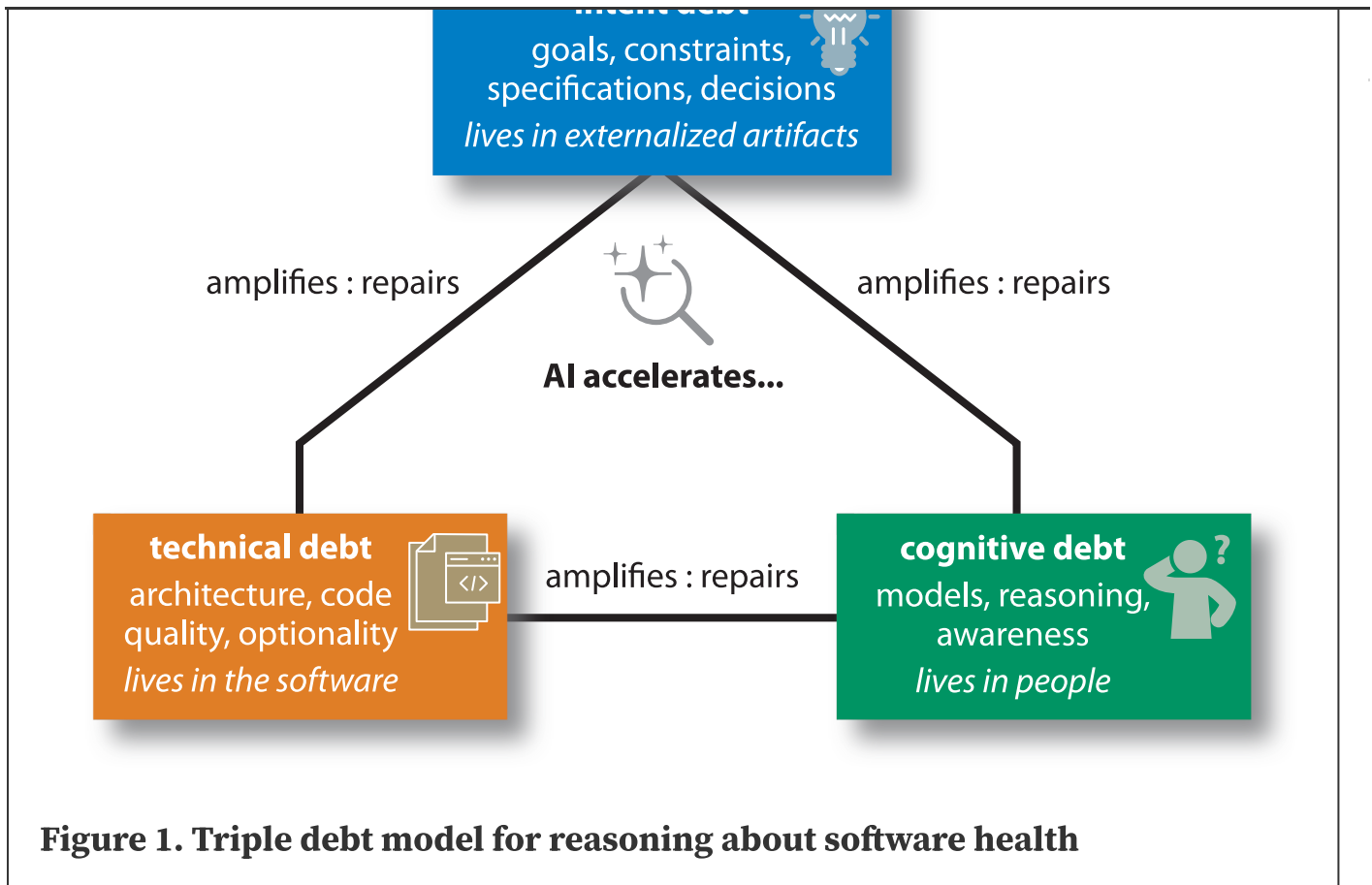
Generative AI (GenAI) is dramatically accelerating the velocity of software development, enabling small teams to ship features at a pace that would have seemed implausible just a few years ago.²⁴ I saw this firsthand in an entrepreneurship course I taught recently. Student teams were building software products over the semester, using GenAI tools to accelerate development, moving quickly to ship features and meet milestones. By week eight, one team hit a wall. People changes were breaking things in unexpected places, and progress had



As we dug deeper, however, a different problem emerged. No one on the team could explain why certain design decisions had been made, or how different parts of the system were supposed to work together. The code might have been messy, but the deeper issue was that the team's shared understanding, the theory of the system,²² had quietly fragmented. They had also failed to write down or communicate the rationale behind decisions. They had accumulated cognitive and intent debt faster than technical debt, and it had paralyzed them.

This is not an isolated story.³³ GenAI does not remove the challenges of software engineering; it redistributes them. This article proposes a triple debt model (see [figure 1](#)) for reasoning about software health, built around three interacting debt types: *Technical debt* refers to problems in the code layer; *cognitive debt* refers to erosion of shared understanding across a team over time; and *intent debt* refers to a lack of externalized goals, constraints, and rationale that both humans and AI systems need to work safely and efficiently with the codebase. Technical debt makes systems harder to change. Cognitive debt makes systems harder to understand. Intent debt makes it difficult to know what the system is for and whether it is meeting real user needs.





The Hidden Cost of AI-Generated Code

For years, software engineers have worried about technical debt,⁶ the long-term cost of messy code that accumulates and compounds when teams prioritize speed over quality. But GenAI might be quietly shifting where the real risk lies. Today, AI systems can generate code faster than developers can read or understand it, and as those models improve, the systems show growing promise for reducing debt through automated refactoring, test generation, and automated code review.¹⁴ That is, GenAI might reduce technical debt while simultaneously accelerating the accumulation of cognitive and intent debt. Organizational expectations for rapid productivity gains, without corresponding investment in learning support, can create a paradoxical effect where developers lack the time to develop the understanding that would actually save them time.²¹

The code might work. It might even be well architected. But the team might not adequately understand how it works or remember why it was built the way it was. Over time, the shared understanding that makes a software system safe to change

What Is a Software System, Really?

When developers think about a software system, they typically think first about code: the files, functions, and architecture that implement its features. But a software system exists across three distinct layers:

- **Goals and intent.** The requirements, constraints, and objectives the system is meant to fulfill, as held by stakeholders and captured in specifications, tests, and documentation.
- **Code and structure.** The representation of a system and the implementation of its intent: the source code, architecture, dependencies, and deployment infrastructure that make the system executable.
- **Shared understanding.** The dynamic mental models that developers, architects, product managers, and other stakeholders hold about how the system works and how it can be reasoned about—what Peter Naur called the “theory of the system.”²² Importantly, no single individual needs to understand all aspects of these layers; what matters is that sufficient shared understanding exists across the team to support safe change and coordination.

Technical debt has long been used to describe problems in the second layer. But the health of a software system depends on the alignment and quality of all three layers. When intent is unclear, systems drift from their intended purpose. When shared understanding is inadequate, teams cannot reason about change safely and are even reluctant to make changes. The system might pass its tests, but the product might have the wrong behavior. These failures are not only about technical debt in the code, although technical debt remains a concern.

Technical Debt: The Well-Known Layer

Technical debt arises when developers make deliberate or inadvertent tradeoffs that prioritize short-term delivery over long-term code quality. They take shortcuts that lead to messy or smelly code and make (or avoid) architectural decisions that constrain future evolution.¹⁸ The argument is that, like financial debt, technical

Technical debt, although problematic, is perhaps the easiest to manage of the three types of debt discussed in this article, in part because of its visibility. And the software engineering community has developed a rich set of accepted and validated practices to manage technical debt:^{2,9} test-driven development, refactoring, and code review, among others. GenAI is increasingly contributing to this work to reduce technical debt, providing automation for refactoring, identification of code or architectural smells, and test generation. This optimism aside, even a codebase with low technical debt can be deeply problematic if the team does not understand it or if it no longer reflects what the system was meant to do.

Cognitive Debt: The Invisible Layer

Cognitive debt is a team-level, project-level property reflecting the erosion of shared understanding across a software system over time. This erosion manifests as increasingly inadequate shared mental models that developers rely on to reason about the system and change it safely and confidently. While individual developers experience this erosion as confusion, loss of control, or reduced confidence,²⁷ the debt itself resides in the accumulation of gaps in shared understanding and awareness across the team. This framing draws on distributed cognition theory, which holds that cognitive processes are properties of systems of people, artifacts, and their interactions rather than of individual minds alone.¹⁵

The term *cognitive debt* has also been used to describe measurable reductions in individual neural engagement during AI-assisted tasks.¹⁷ Related work describes “comprehension debt” as the growing gap between what code developers can produce with AI and what they genuinely understand.¹ The use of the term in this paper, however, focuses on the team-level and longitudinal dimension of software development: the accumulated erosion of shared understanding of a software system over time.

Developers have always struggled to understand not just legacy systems they did not build,³¹ but also software they built themselves using libraries they did not fully grasp or Stack Overflow solutions they copied without comprehending. Classic case studies also observed that software systems depend on distributed human understanding; no one developer understands all of a system.⁷ Software development has never required any one person to understand an entire system;

Cognitive debt is not new; developers have long worked with incomplete and distributed understanding of complex systems. What is new is the rate at which this gap can accumulate and the difficulty of detecting it in AI-assisted development.

GenAI further changes the relative importance of cognitive debt with respect to technical debt in a fundamental way. When a developer writes code from scratch, even messy code, the friction and effort mean they build at least a partial mental model along the way. They strive to understand what the code is trying to do, even if the implementation is imperfect. When AI generates that same code, the developer might accept it without building the same level of understanding.²⁷ At scale, across a team and over time, this creates an accumulation of not knowing across the team. The code works, but the understanding and mental models of how the system behaves and how to reason about it are missing or flawed.

Cognitive Surrender Leads to Cognitive Debt

Reading and reasoning about unfamiliar code are among the most cognitively demanding activities developers undertake,¹² which is partly why understanding erosion is so consequential. The psychological mechanism behind this is what Steven Shaw and Gideon Nave²⁶ of the Wharton School refer to as cognitive surrender: adopting AI outputs with minimal scrutiny, bypassing both intuition and deliberate reasoning (drawing on psychologist Daniel Kahneman's¹⁶ distinction between fast, intuitive System 1 thinking and slower, more deliberate System 2 reasoning).

This is importantly different from cognitive offloading, the strategic delegation of a discrete task to a tool (using a linter, a type checker, a spell checker). Cognitive offloading is a rational productivity choice. Cognitive surrender can also be intentional, but it might also lead to the loss of critical engagement. Even when surrender is intentional, the resulting debt accumulates invisibly. The team doesn't realize what understanding they're losing until it's gone. Notably, Shaw and Nave find that cognitive surrender also inflates confidence even when the AI is wrong, which helps explain why cognitive debt remains invisible until it is too late: The team feels they understand the system better than they do.



on their ideas and cared less about cognitive debt. In so doing, they also accumulated intent debt, failing to maintain a shared understanding of where they were headed. In larger organizations, keeping track of an acceptable amount of cognitive debt can be even more challenging because of its mostly invisible nature and scale.

Effective problem solving requires continuous interaction between a developer's model of the problem and their model of the system, each informing and correcting the other. When AI handles solution building, this feedback loop is severed, leaving developers with an increasingly static understanding of the problem itself,²⁹ while also losing track of who knows what across the team.

Diagnosing Cognitive Debt

Cognitive debt is harder to see and measure than technical debt, because it is distributed across people and not directly visible in the codebase. But it leaves some visible traces and signals:

- **Resistance to change.** Low confidence in understanding the system might make developers across the team reluctant to modify it.²⁷
- **Unexpected results.** A reliable signal of cognitive debt is when a team member makes a change expecting one set of observable outcomes but sees something else entirely.²⁷
- **Slow or unpredictable onboarding.** New team members cannot get up to speed despite documentation, because the documentation describes what the code does, not why, and others on their team also struggle to help newcomers onboard.
- **Loss of transactive memory.**^{11,32} Team members lose track of who knows what and how the system is understood, increasingly relying on individual interactions with AI rather than shared practices.
- **Low bus factor.** Only one person or very few on the team truly understand the system, and other team members are concerned about what might happen if they leave.²⁸

Some of the practices that work for technical debt are also useful here: Human code review, for example, is valuable for not only catching defects, but for spreading understanding across the team. Pair programming serves a similar function. But cognitive debt also calls for specific and new team understanding practices:

- **Design for troubleshooting.** Ensure “diagnostic legibility”²⁰ is preserved, as developers lack familiarity with code they didn’t write but might need to troubleshoot.²⁷
- **Reimplementation to repair cognitive debt.** As the cost of code generation is now relatively “low,” teams can instruct an agent to reimplement a feature using different tests or new design elements that might help rebuild understanding.
- **System walkthroughs.** This is where developers explain code they did not write for the purpose of knowledge sharing and theory building²² rather than documentation or code refactoring.
- **Retrospectives and postmortems.** When things break or challenges emerge, this helps collectively rebuild mental models that might have frayed.
- **Deliberate communication during implementation, onboarding, and offboarding.** This can surface gaps in shared understanding through grounding interactions.⁵

A common theme emerges when considering these practices: The practices that most effectively reduce cognitive debt are those that make implicit knowledge explicit—not as an afterthought but as part of the development process itself. Some practitioners also propose using AI to reduce the cost of these practices and aid in understanding, but they should not fully replace the human friction involved in understanding. The need to externalize motivates the recognition of a third layer of debt, discussed next.

Intent Debt: The Forgotten Layer

There is a third layer of debt that has received less attention, but which becomes increasingly critical as AI systems take on larger roles in software development: Intent debt refers to the absence or erosion of explicit rationale, goals, and



or not captured in any artifact that humans and, increasingly, AI agents can consult. As intent debt accumulates, teams may gradually shift towards optimizing the system and their processes rather than the evolving needs of their users.

What some practitioners are beginning to call *context debt*—the missing information AI agents need to work effectively—often includes symptoms of intent debt. This growing urgency of tools to capture intent is what Marshall McLuhan described as technological “retrieval.”¹⁹ The introduction of new tools often revives practices that earlier technologies made less necessary or even obsolete. As GenAI systems generate more code, practices that capture intent, such as specifications, tests, domain knowledge, and design rationale, might become critical again.

Where technical debt lives in code and cognitive debt lives in people, intent debt lives in incomplete or missing noncode artifacts. Artifacts such as requirements documents, architectural decision records, implementation plans, tests, and specifications are the externalized memory of what a system is supposed to do. When this information is absent, incomplete, fragmented,³⁵ or out of date, intent debt cannot be repaired, and the system might gradually drift from its intended purpose—and neither the team of developers nor the AI agents assisting them will have a reliable source of guidance.

Intent debt is also a familiar challenge in software engineering, especially for larger teams. Software projects have always struggled with requirements that drift, specifications and decisions that are never written down, and goals and constraints that exist only in the minds of a few key stakeholders. Intent is best captured at the moment key decisions are made, as recovering it later can be difficult and sometimes impossible (unlike technical debt, which can be addressed later).

The arrival of AI agents as active participants in software development makes addressing intent debt more urgent than before, both for developers who might have inadequate understanding and for AI agents that might need to refactor, extend, or test a system. AI agents and human developers might need to understand what the system is for, not just what it currently does. Without this information, human developers might find it impossible to build sufficient shared understanding, and agents might optimize for the wrong objectives. Each generation of AI-assisted development not only carries the debt forward, but also compounds it.

Intent debt can be recognized by several patterns:

- **Behavior drift.** The system's behavior diverges from what stakeholders believe it should do, discovered only during early testing or, worse, during customer incidents.
- **AI agents struggle to make changes.** Just as AI agents can struggle with poor-quality code,⁴ they might require extensive clarification, might produce solutions that are technically correct but miss the point, or might use more tokens and time than expected because of a lack of context.
- **Loss of articulated constraints.** Nonfunctional requirements such as performance budgets, privacy constraints, and accessibility requirements are known to a few people and are gradually forgotten.

Practices for Creating and Maintaining Intent Artifacts

Before reliance on GenAI, source code often captured human intent through naming and design. When code is generated by AI, however, intent might need to be deliberately articulated and captured through intent-first workflows. Reducing intent debt requires investing in “living” artifacts that externalize goals, plans, constraints, and reasoning, similar to emerging approaches that treat knowledge itself as a structured, executable artifact:³⁵

- **Executable intent.** Behavior-driven development (BDD) specifications and tests are designed to capture purpose rather than just verify behavior.³⁴ These are intent artifacts that can be executed, and when they fail, they might show that the system has drifted from its intent.
- **Decision and rationale records.** Architectural decision records (ADRs) capture what was decided, why, and what was not done.²³ Domain-driven design (DDD) offers a complementary approach:⁸ Its practices of ubiquitous language and collaborative domain modeling make domain intent explicit before it is encoded in code.
- **Context artifacts for AI-assisted development.** Harness engineering,³ patterns for reducing AI frustration,¹⁰ AI skills, agent instructions, playbooks, and AI-assisted[^]

- **Keep user needs visible.** Make explicit who the system is for, what problem it solves, and what success looks like for its users. Revisit assumptions and success criteria continuously and reflect them in artifacts that guide both developers and AI agents. Validate these regularly against real user feedback and outcomes. Without this, systems may continue to evolve while drifting away from what users actually need.

Despite their promise, these approaches cannot replace the hard human work of deciding what the system is for and ensuring that its evolution remains aligned with the evolving needs of its users.²⁵

Three Layers of Software System Health

Together, technical, cognitive, and intent debt point to a framework for thinking about software system health that goes beyond code quality and changeability:

- **Technical debt lives in code.** It accumulates when implementation decisions compromise future changeability. It limits how systems can change.
- **Cognitive debt lives in people.** It accumulates when a shared understanding of the system erodes faster than it is replenished. It limits how teams can reason about change.
- **Intent debt lives in artifacts.** It accumulates when the goals and constraints that should guide the system are poorly captured or maintained. It limits whether the system continues to reflect what developers meant to build, what users need, and how humans and AI agents can continue to evolve the system effectively.

The three types of debt are system-level properties that interact and reinforce each other. The distributed cognition theory describes how cognitive processes are properties of systems of people, artifacts, and their interactions.¹⁵ Intent debt can cause cognitive debt; when the purpose of a system is not well documented, new and returning team members cannot form accurate mental models of it.

Conversely, developers who lack understanding of the system intent will not be able to externalize specifications and record decisions. Cognitive debt can cause technical debt; when developers do not understand a system, they are more likely to make poor implementation decisions. And technical debt can amplify cognitive

Managing software system health therefore requires proactive attention to all three layers, not just the one that is easiest to measure.

How AI Is Shifting the Balance

As mentioned, the three types of debt are not new, but GenAI is changing their relative importance and the rate at which they accumulate. For managing technical debt, AI shows great potential. Automated refactoring, AI-assisted code review, and AI-generated test suites can reduce technical-debt accumulation and make it easier to improve legacy systems. If current trends continue, some believe AI might increasingly manage the code layer on behalf of developers, reducing the human cost of maintaining code quality.

AI can be used as a partial solution for mitigating cognitive and intent debt, but it might also be a risk multiplier if the humans involved surrender their cognition and do not proactively capture their intent. AI generates code faster than teams can build the understanding needed to safely change it. At least today, it accepts underspecified prompts, fills in gaps, and produces plausible-looking results that might miss the intent or system needs entirely. As AI takes on more of the implementation and documentation work, the feedback loop and friction that traditionally forced developers to understand the code and think about their intent are weakened.

Implications for Practice

What can practitioners do? The framework for system health presented in this article suggests several practical priorities for software teams working with AI-assisted development:

- **Treat understanding as a deliverable.** Just as working code is a product of software development, shared understanding should be treated as a first-class deliverable, something teams invest in explicitly rather than something that happens as a side effect of writing code. This means allocating time for understanding practices such as walkthroughs, retrospectives, and knowledge

- **Intent-first workflows.** Capture intent early when using AI to assist or automate development. ADRs, well-written specifications, domain modeling, decision rationales, plans, and clear user-acceptance criteria are the raw materials that ground human understanding and that AI agents will require to do useful work.
- **Resist the automation of understanding.** There is a temptation to use AI to generate documentation as well as code, producing explanations of what the system does without the team ever building genuine understanding. This substitutes the appearance of understanding for the real thing and makes cognitive debt harder to detect. Teams should be cautious about any practice that produces the artifacts of understanding without hard cognitive work to build important mental models. In the future, the core developer's skill might not be writing code, but maintaining a correct understanding of what the system does and why and how it can evolve.¹³
- **Monitor the three layers in tandem.** Technical debt has a rich ecosystem of tools for measurement and monitoring. Similar attention should be paid to cognitive and intent debt: perhaps through onboarding time tracking, knowledge-concentration metrics, requirements-coverage analysis, and regular audits of the gap between documented intent and actual behavior. At the very least, a team might wish to reflect on which practices they use to minimize debt across the three interacting dimensions mentioned in this article. A focus on only some practices might lead to tradeoffs they do not realize. Software systems depend on intent captured in artifacts, behavior embodied in code, and understanding distributed across the team and its artifacts. When these layers erode or fall out of alignment, the three types of debt accumulate.

Open Questions for Practice

Should we document "intent"?

Not all practitioners agree that capturing intent is worth the effort, arguing that it doesn't matter how they got here; they just need to focus on what changes are needed to go forward and that what they need to know can be

is far from settled.

Can AI help make implicit knowledge explicit?

Some practitioners have proposed that AI can help surface implicit knowledge, reducing intent debt proactively and retroactively. Others counter that the main value in creating documentation is the human understanding gained. Whether and how AI should be used to autogenerate such documentation remains an open question.

Debt as a risk, or debt as a strategy?

This article frames cognitive debt as a risk to be managed, while acknowledging that understanding might not need to be about details of the implementation layer and might be distributed among team members and artifacts. Others argue this understanding is often not needed at all, similarly to how managers have always had to trust team members. With GenAI automating more development work, knowing how much debt is acceptable in a particular organization or project remains uncertain.

Conclusion

GenAI promises extraordinary gains in the speed and ease with which software can be created. As with many technological shifts, however, the benefits might carry a hidden reversal from technical debt to cognitive and intent debt, with a need for tools and processes to support paying that down. The teams and organizations that navigate the AI-assisted development era most successfully will be those who invest in understanding and intent as deliberately as they have invested in code quality. Maintaining healthy software systems in the age of AI might depend on preserving

Acknowledgments

I am grateful to colleagues who provided valuable feedback and ideas reviewing earlier drafts of this work: Adam Tornhill, Kent Beck, Daniel German, Dave Thomas, Marian Petre, Markus Borg, Mary Shaw, Roy Weil, Max Alexander-Kanat, Keith Mann, and Ciara Storey. I am grateful to Arty Starr for discussions that helped sharpen the definitions in this paper and for work that influenced my thinking about how understanding breaks down and is rebuilt in practice. I also thank colleagues who attended the Future of Software Engineering Retreat organized by Thoughtworks in February 2026 for their insights and feedback.

Related Material

DevEx in Action

A study of its tangible impacts

Nicole Forsgren, et al.

<https://queue.acm.org/detail.cfm?id=3639443>

Managing Technical Debt

Shortcuts that save money and time today can cost you down the road

Eric Allman

<https://queue.acm.org/detail.cfm?id=2168798>

Working Models for Tackling Tech Debt

Understand the options to tailor an approach that suits your needs

 Kate Matsudaira



Margaret-Anne Storey is a professor of computer science at the University of Victoria and a Canada research chair in human and social aspects of software engineering. She is coauthor of the SPACE framework and a leading researcher in developer experience (DevEx). Her research focuses on how developers and teams understand complex software systems and how tools, AI, and collaborative practices shape that understanding. Her recent work examines how generative AI is transforming software engineering by changing how understanding is created, shared, and maintained. She collaborates with industry partners including Microsoft and DX. She holds an honorary doctorate from Lund University.

References

- [1] | Alakmeh, T. et al. Grasping AI reliance in program comprehension and coding through the AIRELI persona taxonomy. *IEEE ICPC 2026*, Rio de Janeiro, Brazil. 2026; <https://aireli.hasel.dev/download/aireli-preprint.pdf>

 [Go to Citation](#) |  [Google Scholar](#)

- [2] | Beck, K. *Extreme Programming Explained: Embrace Change*. Addison-Wesley Professional 1999.

 [Go to Citation](#) |  [Digital Library](#) |  [Google Scholar](#)

- [3] | Böckeler, B. Context engineering. martinFowler.com. 2026; <https://martinfowler.com/articles/exploring-gen-ai/context-engineering-coding-agents.html>

 [Go to Citation](#) |  [Google Scholar](#)

- [4] | Borg, M., Hagatulah, N., Tornhill, A., and Söderberg, E. Code for machines, not just humans: quantifying AI-friendliness with code health metrics. 2026. arXiv:2601.02200; <https://arxiv.org/abs/2601.02200>.

 [Go to Citation](#) |  [Google Scholar](#)

[View full text](#) | [Download PDF](#)

[Share](#)

Related articles

Research Article | 23 March 2012

Managing Technical Debt: Shortcuts that save money and time today can cost you down the road.

In 1992, Ward Cunningham published a report at OOPSLA (Object-oriented Programming, Systems, Languages, and Applications) in which he proposed the concept of technical debt. He defines it in terms of immature code: "Shipping first-time code is like going ...

by Eric Allman

Column | 02 July 2024

Working Models for Tackling Tech Debt: Understand the options to tailor an approach that suits your needs

Remember that not all debt is bad, and sometimes, in fact, strategic tech debt can even be used as a valuable tool to achieve certain business goals?just as financial debt can be taken on to obtain capital that can be invested in other profitable ...



Opinion | 10 August 2026

Unethical Ways to Manage Technical Debt: Is ignoring it unethical?

You can ignore it, tie it to unrelated company initiatives, or feign ignorance and simply not offer solutions that add to it. Or... you can adopt more above-board techniques such as Crawl/Walk/Run and the Reverse Farmer's Rule, which reframe projects so ...

by Thomas A. Limoncelli

Research Article | 15 August 2013

Online Algorithms in High-frequency Trading: The challenges faced by competing HFT algorithms

HFT (high-frequency trading) has emerged as a powerful force in modern financial markets. Only 20 years ago, most of the trading volume occurred in exchanges such as the New York Stock Exchange, where humans dressed in brightly colored outfits would ...

by Jacob Loveless, Sasha Stoikov, and Rolf Waeber



02 August 2026

Where to Draw the Line

by Craig A. N. Soules

02 August 2026

Indirected Reality

by Jim Waldo

Business/Management

See all →

09 September 2025

No One Has Time to Work on Your Project: How to work effectively with overwhelmed people to get things done

by Strata Chalup

11 July 2025

In Search of Quietude: Learning to say no to interruption

by George V. Neville-Neil

Development

See all →

10 August 2026

Unethical Ways to Manage Technical Debt: Is ignoring it unethical?

by Thomas A. Limoncelli

19 June 2026



by Fernanda Graciolli and Nada Amin

Open Source

[See all →](#)

19 June 2026

Goodbye, and Thanks for All the Bikesheds!

by Poul-Henning Kamp

23 March 2026

Open Source and the Iceberg Theory

by Alyssa Wright and Stephen Augustus

Security

[See all →](#)

02 April 2026

Security for the Layperson

by Phil Vachon

13 March 2026

On the Evolution of Program State

by Paul Vixie

Web Development

[See all →](#)

03 July 2025

Concurrency in WebAssembly: Experiments in the web and beyond



05 July 2025

When Is WebAssembly Going to Get DOM Support?: Or, how I learned to stop worrying and love glue code

by Daniel Ehrenberg

Categories

Journals
Magazines
Proceedings
Books
SIGs
Conferences
Collections

Subscriptions

Join ACM
Join SIGs
Subscribe to Publications
Institutions and Libraries

About

About ACM Digital Library
Subscription Information
Author Guidelines
Using ACM Digital Library
All Holdings within the ACM Digital Library
ACM Computing Classification System
Accessibility Statement

Connect

-  Contact us via email
-  ACM Queue on Facebook
-  ACM Queue on X
-  ACM on LinkedIn
-  RSS
-  Send Feedback
-  Submit a Bug Report

The ACM Digital Library is published by the Association for Computing Machinery. Copyright © 2026 ACM, Inc.

[Terms of Usage](#) | [Privacy Policy](#) | [Code of Ethics](#)

